Designing a Multi-GPU Pipeline Architecture for Real-Time Digital Inline Holographic Reconstruction

CSCI 8205: Design and Implementation of Multiprocessor Systems

Project Proposal

1. Group Information

Nicholas Bravo-Frank Electrical and Computer Engineering, University of Minnesota

Email: *bravo095@umn.edu*

Anlei Chen Computer Science, University of Minnesota

Email: chen8264@umn.edu

2. Project Description

2.1 Problem Statement

Digital inline holography (DIH) reconstructs three-dimensional particle fields from two-dimensional interference patterns by numerically propagating light through multiple depth planes. A potential complete real-time DIH processing pipeline involves six computational stages with distinct performance profiles: image resize (~1 ms), exponential moving average (EMA) enhancement (~3 ms), multi-plane reconstruction via angular spectrum propagation (~1 ms per depth plane), ML-based particle detection using a CNN (~3-4 ms, batch size 32), ML classification of detected particle crops (~1 ms, batch size 32), and post-processing (~1-10 ms). The six stages are near-balanced in cost, producing a total per-batch latency of approximately 11-13 ms. Beyond this core pipeline, determining each detected particle's precise focal plane requires reconstructing 100-1,000 additional depth planes per particle crop a task that is 10-100× slower than any individual pipeline stage. This creates a two-tier design problem: how should a multi-GPU, multi-core system be organized to (1) run the fast core pipeline at maximum throughput while (2) concurrently draining a computationally expensive autofocus workload without stalling frame ingestion? Or alternatively finding a better method of distributing the multi-plane reconstruction.

2.2 Expected Results

We expect that (1) the near-balanced stage costs will make grouping strategy the dominant factor in throughput, rather than any single bottleneck stage; (2) lock-free ring buffers with GPU-pinned memory will significantly outperform blocking queues due to reduced coherence traffic; (3) GPU reconstruction will achieve an order-of-magnitude speedup over CPU, and the CPU baseline will reveal memory-bandwidth saturation earlier due to the 2D FFT memory access pattern; (4) multi-GPU reconstruction will achieve 8-12× speedup over single-GPU sequential processing, with scaling efficiency >80% up to 4 GPUs for depth-parallel decomposition; and (5) autofocus throughput will be highly sensitive to the GPU allocation strategy, with time-multiplexed scheduling likely outperforming dedicated allocation when the core pipeline has idle cycles between batches.

2.3 Design Contributions

This project designs, implements, and evaluates a holographic processing pipeline architecture targeting a 1-4 GPU workstation. The key design contributions are:

Pipeline stage grouping and multi-GPU co-location. The core pipeline has six stages but only four GPUs, so stages must be grouped. Which stages should share GPUs, and how should memory and compute resources be partitioned? With EMA and ML detection as co-bottlenecks and other stages at 1-2 ms, the grouping strategy directly determines throughput. We design and evaluate multiple stage-to-GPU mappings, analyzing how co-location of compute-similar versus compute-complementary stages affects utilization, inter-GPU transfer overhead, and tail latency.

Custom inter-stage communication mechanism. We design a lock-free ring buffer tailored to holographic image payloads, with cache-line-aligned metadata, GPU-pinned backing memory, and configurable depth to absorb throughput mismatches between stages. We compare this design against blocking queues, double buffering, and CUDA stream-based pipelining.

Asynchronous subsystem design. We design and evaluate asynchronous architectures for decoupling compute-intensive tasks from the real-time pipeline, exploring GPU scheduling and resource allocation strategies under competing workloads.

Architecture comparison. We benchmark holographic reconstruction across GPU (CUDA/cuFFT) and multi-core CPU (OpenMP/FFTW) and analyze the compute-versus-memory-bound behavior on each architecture using hardware performance counters and roofline analysis.

2.4 Evaluation Metrics

The system is evaluated using: (1) throughput in holograms processed per second; (2) per-stage latency distributions (mean, p95, p99); (3) strong and weak scaling efficiency from 1 to 4 GPUs and 1 to 32 CPU cores; (4) cache coherence overhead measured via L1/L2/L3 miss rates and coherence transactions using hardware performance counters; (5) pipeline utilization as the fraction of time each GPU is active; and (6) tail latency ratio (p99/p50) to characterize real-time predictability. For the autofocus subsystem, we additionally measure crops focused per second and autofocus queue backlog depth over time.

3. Related Work

3.1 Digital Inline Holography for Particle Detection

Li et al. [1] introduced novel deep learning methods specifically for DIH reconstruction using auto-encoders for blind single-shot hologram reconstruction. This work demonstrates that deep learning can achieve superior reconstruction quality compared to traditional analytical methods while reducing computational overhead. The tutorial by Berg et al. [2] provides comprehensive coverage of aerosol characterization using DIH, focusing on contact-free determination of particle shape, size, and concentration, the core application domain of our pipeline. Recent work by Sanborn et al. [3] demonstrates ML-assisted DIH for real-time detection and classification of biological particles with significantly reduced processing time compared to traditional approaches.

3.2 Real-Time DIH Systems and GPU Acceleration

The Holovibes system [4] represents the current state-of-the-art in real-time holographic processing, achieving 71,400 fps hologram rendering at 256×256 pixels with 30ms end-to-end latency using CUDA-based GPU acceleration, multithreaded parallelism, and efficient buffering. This system implements multiple forms of parallelization directly relevant to our design: acquisition, rendering, and recording tasks executed in parallel; computations offloaded to GPU; and concurrent CPU-GPU data transfers during ongoing computations. Wang et al. [5] introduced novel DIH systems for simultaneous characterization and classification of airborne particles using modern GPU acceleration techniques, demonstrating real-time capabilities for aircraft icing studies.

3.3 Parallel FFT and GPU Optimization

The core computational bottleneck in holographic reconstruction is the 2D FFT. NVIDIA's cuFFT library supports single-GPU and multi-GPU (up to 16) FFT computation, while cuFFTDx provides device-side extensions for kernel fusion with >2× speedup [6]. Ayala et al. [7] analyzed distributed 3D FFT performance on multi-GPU systems, comparing slab versus pencil decomposition and demonstrating approximately 90% scaling efficiency. Recent advances in spatial and Fourier neural operators achieve propagation-adaptive holographic synthesis with 18.6% GPU memory reduction and 2.6× speedup over previous approaches [8].

3.4 ML-Based Holographic Particle Analysis

Deep learning integration in holographic particle analysis has seen significant advancement. The CATCH system [9] demonstrated real-time holographic particle characterization with nanometer precision using deep neural networks. Chen et al. [10] proposed model-based deep networks for 3D holographic particle imaging, while Shao and Mallery [11] achieved sub-voxel z-positioning accuracy using U-net architectures. These ML approaches are directly relevant to our detection and classification pipeline stages.

3.5 Lock-Free Synchronization and Pipeline Optimization

Giacomoni et al. [12] demonstrated that locking queue overhead exceeds 60% of per-stage processing time and proposed the FastForward cache-optimized lock-free queue. Aldinucci et al. [13] developed efficient SPSC queues for fine-grained parallelism, while the moodycamel::ConcurrentQueue [14] provides production MPMC implementation using per-producer sub-queues. For multi-DNN scheduling, DART [15] provides deterministic response time across multiple models sharing GPU resources, addressing the multi-model scheduling problem relevant to running reconstruction, detection, and classification concurrently.

3.6 Research Gap

While recent work demonstrates significant progress in DIH for particle detection [1-3], real-time processing [4], and GPU acceleration [5,8], several critical gaps remain for practical high-throughput systems. First, existing real-time DIH systems focus on single computational tasks (reconstruction only) rather than complete multi-stage processing pipelines that integrate preprocessing, reconstruction, ML inference, and post-processing. The Holovibes system [4] achieves impressive frame rates but targets holographic rendering, not the heterogeneous workload profile of measurement pipelines with competing computational demands. Second, no prior work addresses the fundamental resource allocation problem of multi-stage DIH pipelines on multi-GPU systems. The challenge of optimally grouping six heterogeneous computational stages onto four GPUs while minimizing communication overhead has not been systematically studied. Existing multi-GPU work focuses on scaling single algorithms [7] rather than pipeline-level optimization. Finally, the 10-100× computational disparity between core pipeline stages and autofocus creates unique scheduling and resource contention challenges not addressed by existing multi-DNN scheduling frameworks [15].

4. Work Plan

Core Components (Weeks 1-4). Port angular spectrum propagation to C++/CUDA with cuFFT and validate against our existing Python implementation. Implement both depth-parallel (slice decomposition) and spatial-parallel (tile decomposition) multi-GPU strategies for reconstruction. Implement OpenMP

CPU baseline using FFTW on the 32-core Threadripper, measuring strong scaling from 1 to 32 cores. Profile both implementations using Nsight Compute (per-kernel) and perf (cache counters) to construct roofline analysis characterizing compute vs. memory-bound regimes. Deliverable: validated reconstruction kernels on both architectures with initial roofline plots.

Pipeline Assembly and Stage Grouping (Weeks 5-8). Implement the full 6-stage pipeline in C++ with each stage running on dedicated threads. Design and evaluate multiple stage-to-GPU groupings: bottleneck-isolated (each co-bottleneck on its own GPU), compute-balanced (equalizing total per-GPU compute time), and preprocessing/inference split. Deliverable: running multi-GPU pipeline with per-grouping utilization measurements.

Synchronization and Asynchronous Subsystem Design (Weeks 9-12). Implement four inter-stage communication mechanisms: blocking queue (mutex + condition_variable), lock-free SPSC ring buffer with cache-line-aligned metadata, double/triple buffering, and CUDA stream pipelining with event-based dependencies. Design the asynchronous autofocus subsystem with a work-stealing queue feeding dedicated GPU workers. Compare three GPU allocation strategies: dedicated (reserve GPUs exclusively for autofocus), time-multiplexed (use core pipeline idle cycles), and depth-parallel (spread one crop's planes across multiple GPUs). Deliverable: synchronization benchmark results and autofocus subsystem with queue depth monitoring.

Analysis and Evaluation (Weeks 13-16). Conduct cache coherence analysis using perf/likwid to measure false sharing and NUMA effects. Perform strong/weak scaling analysis and Amdahl's law characterization. Evaluate real-time feasibility against a 400 Hz camera frame rate target, characterizing p50/p95/p99/p99.9 latency. Investigate kernel fusion opportunities using cuFFTDx to merge FFT with pre/post-processing. Deliverable: comprehensive performance report with architecture diagrams, Nsight timelines, roofline plots, latency CDFs, and GPU resource allocation recommendation.

5. Schedule

Week	Dates	Tasks
1-2	Feb 11-22	C++/CUDA reconstruction kernel; validate against Python reference; benchmark across image sizes and depth counts
3-4	Feb 23-Mar 8	OpenMP CPU baseline (1-32 cores); roofline analysis; initial 6-stage pipeline with blocking queues; initial performance profiling; Milestone 1 report
5-6	Mar 9-22	ML inference integration (ONNX/TorchScript); stage-to-GPU grouping design; profile 3 grouping strategies
7-8	Mar 23-Apr 5	Lock-free ring buffer and 3 alternative synchronization mechanisms; autofocus subsystem with work-stealing queue; Milestone 2 report
9-10	Apr 6-19	Combined system: core pipeline + autofocus under resource contention; cache coherence and NUMA analysis; false sharing experiments
11-12	Apr 20-30	Strong/weak scaling analysis; real-time feasibility evaluation at 400 Hz; kernel fusion (cuFFTDx); final report

References

[1] Li, H., Chen, X., Chi, Z., Mann, C., & Razi, A. (2020). Deep DIH: single-shot digital in-line holography reconstruction by deep learning. *Ieee Access*, *8*, 202648-202659.

[2] Berg, M. J. (2022). Tutorial: Aerosol characterization with digital in-line holography. *Journal of Aerosol Science*, *165*, 106023.

[3] Sanborn, D., He, R., Feng, L., & Hong, J. (2023). In situ biological particle analyzer based on digital inline holography. *Biotechnology and bioengineering*, *120*(5), 1399-1410.

[4] Dubosc, M., Boy-Arnould, M., Guillou, J., Gragnic, T., Courselle, A., Hervé, G., ... & Atlan, M. (2025). Holovibes: real-time ultrahigh-speed digital hologram rendering and short-time analysis. *arXiv preprint arXiv:2508.03911*.

[5] Wang, J., Chumbley, E., Shyam Kumar, M., Hong, J., & Hu, H. (2025). Development of a digital inline holographic system for aircraft icing studies. *Applied Optics*, *64*(16), D95-D102.

[6] NVIDIA, "cuFFT / cuFFTDx / cuFFTMp documentation," NVIDIA Developer, 2025.

[7] Ayala, A., Tomov, S., Stoyanov, M., Haidar, A., & Dongarra, J. (2022, May). Performance analysis of parallel FFT on large multi-GPU systems. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (pp. 372-381). IEEE.

[8] Liu, N., Liu, K., Yang, Y., Peng, Y., & Cao, L. (2025). Propagation-adaptive 4K computer-generated holography using physics-constrained spatial and Fourier neural operator. *Nature Communications*, *16*(1), 7761.

[9] Altman, L. E., & Grier, D. G. (2020). CATCH: Characterizing and tracking colloids holographically using deep neural networks. *The Journal of Physical Chemistry B*, *124*(9), 1602-1610.

[10] Chen, N., Wang, C., & Heidrich, W. (2021). Holographic 3D particle imaging with model-based deep network. *IEEE Transactions on Computational Imaging*, *7*, 288-296.

[11] Shao, S., Mallery, K., Kumar, S. S., & Hong, J. (2020). Machine learning holography for 3D particle field imaging. *Optics Express*, *28*(3), 2987-2999.

[12] Giacomoni, J., Moseley, T., & Vachharajani, M. (2008, February). Fastforward for efficient pipeline parallelism: a cache-optimized concurrent lock-free queue. In *Proceedings of the 13th ACM SIGPLAN Symposium on Principles and practice of parallel programming* (pp. 43-52).

[13] Aldinucci, M., Danelutto, M., Kilpatrick, P., Meneghin, M., & Torquati, M. (2012, August). An efficient unbounded lock-free queue for multi-core systems. In *European Conference on Parallel Processing* (pp. 662-673). Berlin, Heidelberg: Springer Berlin Heidelberg.

[14] C. Desnoyers, "moodycamel::ConcurrentQueue," GitHub, 2024. [Online]. Available: https://github.com/cameron314/concurrentqueue

[15] Xiang, Y., & Kim, H. (2019, December). Pipelined data-parallel CPU/GPU scheduling for multi-DNN real-time inference. In *2019 IEEE Real-Time Systems Symposium (RTSS)* (pp. 392-405). IEEE.